

ACCENTURE

Complete Preparation Guide

Basics to Advanced • 2027 Batch

Online Assessment • Cognitive Games • Technical MCQs
Coding Round • Communication Test • Interview

All Rounds Covered

PYQs + Solutions

Cheat Sheets

Compiled from the official SRMIST preparation deck by
Dr. S. Girirajan, Assistant Professor, Department of Computing Technologies,
School of Computing, SRMIST

Expanded with current (2026–27) drive patterns from public preparation sources.
Pattern varies by drive and region — always confirm with your placement cell / invite mail.

Contents

#	Chapter	What you get
0	How to Use This Guide + 30/15/7 Day Plans	Strategy
1	Hiring Process, Eligibility, Tracks & Cut-offs	Overview
2	Round 1A — Behavioural Assessment	Theory + 60 statements with answers
3	Round 1B — Cognitive Assessment (Gamified)	Maze, Bubbles, Path Finder
4	Cognitive Backup — Classic Aptitude	Formula sheet + shortcuts
5	Round 2 — Technical MCQ: Pseudocode	Full rules + 25 solved PYQs
6	Round 2 — MS Office & Common Applications	Cheat notes + PYQs
7	Round 2 — OOPs Concepts	Cheat notes + PYQs
8	Round 2 — DBMS & SQL	Cheat notes + PYQs
9	Round 2 — Operating Systems, CPU & Memory	Cheat notes + PYQs
10	Round 2 — Networking, Security, Cloud, Wi-Fi, DevOps	Cheat notes + PYQs
11	Round 3 — Coding: DSA Patterns & Solved Problems	20+ problems, full code
12	Round 3 — SQL Query Question	Deck question solved + drills
13	Round 3 — Web Development (HTML/CSS/JS)	Deck question solved + drills
14	Communication Assessment	Section-wise strategy
15	Technical Interview — Q&A Bank	50 questions with model answers
16	HR Interview — Q&A Bank	25 questions with model answers
17	Master Cheat Sheet (print this)	One-page revision
18	Exam-Day Playbook & Common Mistakes	Tips & tricks

0. How to Use This Guide

This guide is built around one fact: **Accenture eliminates most candidates in the online test, not in the interview.** The test is not hard — it is *wide and fast*. You lose marks to unfamiliarity and clock pressure, not to difficulty.

The 80/20 of Accenture prep. If you only have limited time, master these five things: (1) Pseudocode output tracing, (2) SQL joins + GROUP BY + HAVING, (3) One coding language end-to-end (Python or Java), (4) OOPs + OS + Networking one-liners, (5) Speaking clearly for 30 seconds without pausing. That covers roughly 80% of the marks.

30-Day Plan (ideal)

Days	Focus	Daily output target
1-4	Pseudocode: operators, loops, arrays, bitwise, recursion	30 trace questions/day
5-8	OOPs + DBMS/SQL theory	40 MCQs + 10 SQL queries/day
9-12	OS + CPU/Memory + MS Office	40 MCQs/day
13-16	Networking, Security, Cloud, Wi-Fi, DevOps	40 MCQs/day
17-22	Coding: arrays, strings, math, hashing	3 problems/day, timed 20 min each
23-25	SQL coding + JS DOM coding	5 queries + 2 JS tasks/day
26-27	Cognitive games + mental math speed	15 min bubble math, 15 min maze
28	Communication test practice (record yourself)	60 min speaking
29-30	2 full mocks + revise this guide's cheat sheet	Mock + review

15-Day Plan (realistic)

Days 1-3 Pseudocode • 4-5 OOPs + SQL • 6-7 OS + Networking/Cloud/Security • 8-11 Coding (arrays, strings, math, SQL, JS) • 12 MS Office + DevOps + Wi-Fi • 13 Behavioural + Cognitive games • 14 Communication + Interview Q&A • 15 Full mock + cheat sheet revision.

7-Day Emergency Plan

Day 1 Pseudocode (all day) • Day 2 SQL + DBMS • Day 3 OOPs + OS • Day 4 Networking + Cloud + Security + MS Office • Day 5 Coding practice (10 problems) • Day 6 JS/Web + Cognitive games + Behavioural • Day 7 Cheat sheet + interview Q&A + speak English aloud for 1 hour.

Tip: Do not "read" this guide. Cover the answer, attempt, then check. Passive reading of MCQs creates false confidence — the single biggest reason strong students fail Accenture's technical section.

1. Hiring Process, Eligibility & Cut-offs

1.1 The Three Assessment Stages

Per the SRMIST deck, Accenture hiring has three assessment rounds:

1. **Online Written Test** — Behavioural, Cognitive, Technical, Coding.

ELIMINATION
2. **Communication Assessment** — AI-graded spoken/written English.
3. **Interview Round** — Technical + HR.

1.2 Online Written Test Structure

Total: 125 questions • 175 minutes • 4 sections

Round	Section	Questions	Time	Nature
1A	Behavioural Assessment	54	20 min	Personality — no right/wrong
1B	Cognitive Assessment	3–4 games (24 sub-items in bubble math)	20 min	Game-based reasoning
2	Technical Assessment (MCQ)	45	45 min	CS fundamentals
3	Coding Assessment	3	60 min	DSA + SQL + Web

Important: Round timings are **sectional and non-reversible**. You cannot go back to a finished section, and leftover time in one section does **not** carry into the next. Budget per question, not per test.

1.3 Technical Assessment Syllabus (45 Q / 45 min)

#	Topic	Typical weight
1	Common Applications and MS Office	5–8 Q
2	Pseudocode	10–15 Q (highest)
3	Network Security and Cloud	5–8 Q
4	Wi-Fi Security	2–4 Q
5	DevOps	2–4 Q
6	OOPs	4–6 Q
7	SQL	4–6 Q
8	Operating System	3–5 Q
9	CPU, Memory	2–4 Q

1.4 Coding Round (3 Q / 60 min)

- **Q1** — DSA / Basic Programming in Java, Python or C#
- **Q2** — SQL query-based question
- **Q3** — Web-based question (HTML / CSS / JavaScript)

Suggested split: 22 min DSA • 15 min SQL • 18 min Web • 5 min buffer.

1.5 Eligibility (typical fresher drive)

Criterion	Usual requirement
Degree	BE/BTech/ME/MTech/MCA/MSc (branch-dependent per drive)
Academics	60% or 6.0–6.5 CGPA throughout 10th, 12th, UG (varies)
Backlogs	No active/standing backlogs at time of joining
Gap	Usually max 1 year, education gap must be explainable
Mobility	Must be open to any Accenture India location & any shift
Bond/Agreement	Typically a service agreement/training period — read your offer

1.6 Two Hiring Tracks

Accenture typically evaluates two fresher tracks in the same sitting; **your coding score usually decides the track.**

Track	Indicative package	What pushes you here
Associate Software Engineer (ASE)	~4.5–6.5 LPA	Clear all sectional cut-offs

Advanced ASE (11A grade)	~6.5-9.0 LPA	Strong coding round + technical depth
--------------------------	--------------	---------------------------------------

Trick: You do not apply twice. Attempt every coding question — even a partially passing solution can lift you into the Advanced track. Never leave Q2/Q3 blank.

1.7 Realistic Cut-off Guidance

Section	Safe target	Notes
Behavioural	Answer all 54	Consistency matters more than score
Cognitive games	Solve with fewest attempts	Process is scored, not just answer
Technical MCQ	28-32 / 45	Sectional cut-off exists
Coding	1 full + 1 partial minimum	2 full = Advanced ASE range
Communication	Speak fluently, no long pauses	AI-graded, no human listener

These are community-reported ranges, not official numbers. Treat them as targets, not guarantees.

2. Round 1A — Behavioural Assessment

54 questions • 20 minutes • ~22 seconds per statement. You rate statements on a 5-point scale: **Highly Agree / Agree / Neutral / Disagree / Highly Disagree**. No technical questions appear here.

2.1 What is actually being measured

Trait	What Accenture wants to see	Red flag they look for
Thought Process	Structured, logical, evidence-based decisions	Impulsiveness
Leadership Skills	Taking ownership, guiding without dominating	Passivity or bossiness
Collaboration Skills	Works well with different personality types	Preferring to work alone always
Problem Solving	Breaks problems down, tries alternatives	Giving up / escalating instantly
Continuous Learning	Learns new tech willingly	Comfort-zone attachment
Business Ethics	Honesty even when costly	Hiding mistakes, shortcuts
Emotional Resilience	Calm under deadline and criticism	Defensiveness, stress avoidance

2.2 The Five Golden Rules

- Be consistent.** The same trait is tested 4-6 times with reworded and reversed statements. Contradicting yourself is the #1 way to get flagged.
- Avoid "Neutral".** Overusing Neutral reads as evasive or indecisive. Use it 3-5 times at most across 54 questions.
- Do not be a robot saint.** Marking "Highly Agree" on all 54 triggers a social-desirability flag. Mix Agree and Highly Agree.
- Answer fast.** 20 min / 54 Q. Do not analyse — your first instinct, filtered through "what would a good colleague do", is right.
- Watch for reversed wording.** "I prefer to keep mistakes to myself" is the reverse of accountability. Read the verb, not the vibe.

Common trap: Statements containing *always, never, everyone, only, exactly the same, refuse to* are usually extreme. Extreme positive statements deserve Agree (not Highly Agree); extreme negative statements deserve Highly Disagree.

2.3 Worked Examples (from the SRMIST deck)

Statement	Best answer	Trait tested
I remain calm and focused when I have to complete an important task within a short deadline.	Highly Agree	Pressure Handling
I prefer working with people who think exactly the same way I do.	Disagree	Teamwork / Adaptability
When I receive constructive feedback, I use it to improve my future performance.	Highly Agree	Feedback / Growth Mindset
If I make a mistake at work, I prefer to keep it to myself unless someone notices it.	Highly Disagree	Accountability / Integrity
I am comfortable learning a new technology even when I have little prior experience with it.	Highly Agree	Adaptability / Learning

2.4 Handling Pressure — 5 Questions (deck set, with answer key)

#	Statement	Answer
1	I remain calm and focused when working under tight deadlines.	Agree
2	When several tasks are due at the same time, I find it difficult to decide which one to prioritize.	Disagree
3	I can maintain the quality of my work even when I am under significant pressure.	Highly Agree
4	When a task becomes stressful, I prefer to postpone it until the pressure is reduced.	Highly Disagree
5	I organize my work effectively when I have multiple competing priorities.	Highly Agree

Deck answer key: 1. Agree 2. Disagree 3. Highly Agree 4. Highly Disagree 5. Highly Agree

2.5 Practice Bank — 60 Statements with Recommended Responses

HA = Highly Agree, A = Agree, N = Neutral, D = Disagree, HD = Highly Disagree. Cover the right column and attempt first.

A. Thought Process & Decision Making

#	Statement	Ans
1	I gather relevant facts before forming an opinion on a problem.	HA
2	I usually make important decisions based on gut feeling alone.	D
3	I like to consider more than one solution before choosing an approach.	HA

4	Once I decide something, I rarely revisit it even if new information appears.	HD
5	I can explain the reasoning behind my decisions to others.	HA
6	Analysing details slows me down too much to be useful.	D
7	I break large problems into smaller manageable parts.	HA
8	I am comfortable making a decision with incomplete information when a deadline requires it.	A

B. Leadership & Ownership

#	Statement	Ans
9	I take responsibility for the outcome of tasks assigned to me.	HA
10	If my team misses a deadline, I look at what I could have done differently.	HA
11	I wait for someone else to take charge when a group has no clear leader.	D
12	I am comfortable making an unpopular decision if it is the right one.	A
13	Good leaders make all the decisions themselves.	HD
14	I willingly help a teammate finish their work even if it is not my responsibility.	HA
15	I find it difficult to delegate work to others.	D
16	I give credit to team members publicly when work goes well.	HA

C. Collaboration & Teamwork

#	Statement	Ans
17	I enjoy working with people from different backgrounds and skill levels.	HA
18	I prefer to work alone because teams slow me down.	HD
19	I listen fully to a colleague's idea before responding to it.	HA
20	Disagreements within a team usually damage the outcome.	D
21	I adjust my communication style depending on who I am talking to.	HA
22	I find it hard to trust work done by other people.	D
23	I share useful information with my team instead of keeping it to myself.	HA
24	I stay silent in meetings even when I disagree with the plan.	D

D. Problem Solving

#	Statement	Ans
25	When my first approach fails, I try a different one rather than giving up.	HA
26	I escalate a problem to my manager as soon as I hit any difficulty.	D
27	I look for the root cause rather than applying a quick fix.	HA
28	Unfamiliar problems make me anxious rather than curious.	D
29	I test my solution before declaring the task complete.	HA
30	I ask for help after making a genuine attempt on my own.	HA
31	I prefer problems that have one clearly defined correct answer.	N
32	I document what I learn from solving a difficult problem.	A

E. Continuous Learning & Adaptability

#	Statement	Ans
33	I learn new tools on my own time when my role requires them.	HA
34	Changes to project requirements frustrate me more than they motivate me.	D
35	I am comfortable working on a technology I have never used before.	HA
36	I prefer to stay with processes I already know well.	D
37	I actively seek feedback rather than waiting for a review.	HA
38	Being moved to a different team or project would unsettle me for a long time.	HD
39	I read about developments in my field outside of work hours.	A
40	I see mistakes as useful information rather than failures.	HA

F. Business Ethics & Integrity

#	Statement	Ans
---	-----------	-----

41	I report my own errors immediately even if nobody has noticed them.	HA
42	Bending a small rule is acceptable if it helps the client.	HD
43	I follow data confidentiality rules strictly, without exception.	HA
44	If I saw a colleague falsify a report, I would raise it through the proper channel.	HA
45	Meeting the target matters more than how the target is met.	HD
46	I would accept a small gift from a vendor I am evaluating.	HD
47	I keep my commitments even when it becomes inconvenient.	HA
48	I tell a client the realistic timeline even if they want to hear a shorter one.	HA

G. Emotional Resilience & Pressure

#	Statement	Ans
49	Criticism of my work does not affect my performance the next day.	A
50	I keep working effectively when priorities change suddenly.	HA
51	I take setbacks personally.	D
52	I can separate my emotions from a professional disagreement.	HA
53	I need a long recovery period after a stressful project.	D
54	I stay composed when a client is unhappy with our work.	HA
55	Working night shifts or rotational shifts would be a serious problem for me.	D
56	I maintain my work quality even during long stretches of repetitive work.	A
57	I lose motivation when results take a long time to appear.	D
58	I plan my week so that unexpected work does not derail me.	HA
59	I complain to colleagues when management makes a decision I dislike.	D
60	I recover quickly after a project I worked hard on is cancelled.	A

Trick that works: Before starting, fix a persona in your head — "*calm, honest, curious team player who takes ownership and adapts.*" Answer every statement as that person. This single move guarantees internal consistency across all 54 items.

3. Round 1B — Cognitive Assessment (Game-Based)

3-4 games • 20 minutes. The traditional Quant + Logical + Verbal format has been replaced on many 2026-27 drives by gamified reasoning. Note that rollout is uneven — some drives still use the classic sectioned test, so Chapter 4 covers that as backup.

3.1 Game 1 — Memory Maze (Lock and Door)

The rules, exactly as stated

- You move between boxes in a grid containing a maze of **invisible walls**.
- Movement is **up, down, left, right** — **never diagonal**. You may retrace your own path.
- Each time you hit a wall you are **returned to the start** and must begin again. **The walls do not move** — you must remember where they are.
- Goal: collect the **key**, then reach the **door**, in the **fewest attempts**.
- You must reach the key **before** entering the door.
- No need to rush; if unsolved within the time limit you progress automatically to the next maze. A timer sits at the bottom of the screen.

Winning strategy

1. **Number the grid mentally.** For a 3×3 grid label cells A1 A2 A3 / B1 B2 B3 / C1 C2 C3. Never think in "left/right" — think in coordinates. This is what makes walls memorable.
2. **Build a wall map, not a path.** Every collision is free information. After a failure, immediately verbalise: "*wall between B2 and B3*". You are not failing, you are mapping.
3. **Probe outward from the start.** On attempt 1, deliberately test the cell adjacent to the key. Cheap failures early beat expensive failures late.
4. **Replay the known-good prefix fast, then probe one new step.** Classic incremental search: safe path + one experiment per attempt. Never experiment twice in the same attempt.
5. **Prefer the perimeter.** Edge routes have fewer possible wall configurations than centre routes in small grids.
6. **Key first, always.** Walking onto the door before the key wastes an attempt even if the route was valid.

Worked example (deck grid)

Grid 3×3. Key at **A2** (top-middle). Person at **C1** (bottom-left). Door at **C3** (bottom-right).

	col1	col2	col3
row A	.	KEY	.
row B	.	.	.
row C	PERSON	.	DOOR

Shortest theoretical route: C1 → B1 → A1 → A2 (key) → B2 → C2 → C3 (door) = 6 moves. Alternative of equal length: C1 → B1 → B2 → A2 (key) → B2 → C2 → C3.

Approach: Attempt 1 — try the 6-move route. If blocked at, say, B1→A1, record that wall and on attempt 2 take C1 → B1 → B2 → A2 instead. After the key, the return leg B2 → C2 → C3 is already partially verified from your outbound trip, so reuse it.

Scoring insight: The metric is attempts, not seconds. Moving slowly and remembering is strictly better than moving fast and re-colliding. Most candidates lose here by panicking at the timer.

3.2 Game 2 — Maths Puzzle / Bubble Sort Challenge

24 questions • each auto-advances after ~14 seconds. Three (sometimes four) bubbles each contain an arithmetic expression. You must order them — lowest to highest, or highest to lowest, as instructed.

The #1 mistake: Candidates read the expressions correctly but miss the **direction** of the sort. The instruction flips between questions. Read the instruction line *first*, every single time.

Deck example — the three-bubble layout

Bubbles shown: 9 + 13, 7 + 7, 14 + 10. Values: 22, 14, 24.

Lowest → highest: **7+7 (14), 9+13 (22), 14+10 (24)**. Highest → lowest: 14+10, 9+13, 7+7.

Solved PYQ set

Q	Instruction	Bubbles	Values	Answer order
1	Lowest to highest	4−5 • 11−13 • 0+3	−1, −2, 3	11−13, 4−5, 0+3
2	Highest to lowest	6×4 • 30÷2 • 9+9	24, 15, 18	6×4, 9+9, 30÷2
3	Lowest to highest	7×3 • 45÷5 • 12+6	21, 9, 18	45÷5, 12+6, 7×3
4	Highest to lowest	2×2×2 • 100÷20 • 3+4	8, 5, 7	2×2×2, 3+4, 100÷20
5	Lowest to highest	−3×4 • 8−20 • 2−2	−12, −12, 0	−3×4 = 8−20, then 2−2 (tie)

6	Highest to lowest	$15\% \text{ of } 200 \cdot 5 \times 7 \cdot 90 \div 3$	30, 35, 30	5×7 , then the two 30s
7	Lowest to highest	$9^2 \cdot 4^3 \cdot 2^4$	81, 64, 16	2^4 , 4^3 , 9^2
8	Highest to lowest	$144 \div 12 \cdot 3 \times 5 \cdot 20 - 9$	12, 15, 11	3×5 , $144 \div 12$, $20 - 9$
9	Lowest to highest	$17 - 25 \cdot -2 \times 3 \cdot 1 - 1$	-8, -6, 0	$17 - 25$, -2×3 , $1 - 1$
10	Highest to lowest	$12 \times 8 \cdot 140 \div 2 \cdot 15 \times 6$	96, 70, 90	12×8, 15×6, $140 \div 2$
11	Lowest to highest	$25\% \text{ of } 80 \cdot 6 + 6 \cdot 40 \div 8$	20, 12, 5	$40 \div 8$, $6 + 6$, $25\% \text{ of } 80$
12	Highest to lowest	$11 \times 11 \cdot 10^2 \cdot 13 \times 9$	121, 100, 117	11×11 , 13×9 , 10^2

Speed techniques (drill these)

- **Estimate before computing.** If the three values are 96, 70, 90 you do not need exact answers for $140 \div 2$ — you need only relative size. Rounding is enough 80% of the time.
- **Sign first.** Any negative result instantly beats any positive result in a lowest-to-highest sort. Scan for minus-dominant expressions before doing arithmetic.
- **Squares 1-25 and cubes 1-12 by memory.** These appear constantly and cost 4 seconds each if you compute them.
- **Multiplication by 11:** $13 \times 11 \rightarrow 1_ (1+3)_ 3 = 143$. By 5: halve then $\times 10$. By 9: $\times 10$ minus itself.
- **Percentages:** $15\% = 10\% + 5\%$. $25\% = \text{quarter}$. $12.5\% = \text{eighth}$.
- **Never leave blank.** Wrong order scores zero, same as blank — but a guess has 1/6 chance on three items. Always submit something before the 14s auto-advance.

3.3 Game 3 — Path Finder (Rocket / Tile Routing)

Format seen in the deck: **Level 1/5**, a countdown (~04:38), a 3×3 board of tiles each showing arrow segments, a rocket on the left and Earth on the right. Controls: **Rotate**, **Flip**, **Launch**.

How it works

Each tile contains a fixed arrow path. Rotating turns the tile 90°; flipping mirrors it. You must arrange tiles so the arrows form one continuous connected route from the rocket (entry, left edge) to Earth (exit, right edge). Then press Launch. A failed launch costs an attempt.

Strategy

1. **Work backwards from the exit.** The Earth tile position fixes the required exit direction of the rightmost tile. There are usually far fewer valid right-column configurations than left-column ones.
2. **Identify entry and exit ports per tile.** Before rotating anything, note for each tile which of its four edges the arrows touch. A tile with arrows touching only left+bottom can never form a straight horizontal run.
3. **Rotate mentally, click physically.** Every rotation click costs a second. Decide the target orientation in your head, then execute in the minimum number of clicks (rotate is cyclic — 3 clicks clockwise = 1 anticlockwise if available).
4. **Flip is for chirality, not direction.** Use flip only when an L-bend curves the wrong way; rotation alone cannot fix a mirrored bend.
5. **Verify the whole chain before Launch.** Trace the route with your finger on screen. One failed launch teaches you less here than in the maze, because nothing is hidden — the information was already visible.
6. **Levels get harder (1/5 → 5/5).** Bank time on early levels; do not perfect a level you have already solved.

Practice equivalents: the mobile games *Pipe/Plumber puzzles*, *Flow Free*, and *Rush Hour*. Twenty minutes a day for a week is enough to make this round feel routine.

3.4 Other Game Types Reported on Recent Drives

Game	What it looks like	Skill tested
Recall / N-back	A sequence of shapes or numbers flashes; you report what appeared N steps ago	Working memory
Balloon / Risk task	Pump to earn points; bank before it bursts	Risk calibration — be moderately, consistently cautious
Card / Rule inference	Cards give rewards under a hidden rule that changes	Adaptability, learning from feedback
Grid pattern matching	Pick the figure that completes an abstract sequence	Abstract reasoning
Reaction / Go-No-Go	Tap on target stimulus, withhold on distractor	Attention and impulse control

Universal rule for all gamified rounds: these tests score **behavioural consistency**, not just correctness. Erratic play (very fast then very slow, reckless then timid) scores worse than steady, moderately careful play with a few honest mistakes.

4. Cognitive Backup — Classic Aptitude Cheat Sheet

Some 2026–27 drives still run the older sectioned cognitive test: **English Ability (~17 Q), Critical Reasoning & Problem Solving (~18 Q), Abstract Reasoning (~15 Q)**. Prepare this as insurance — the underlying skills also power the games.

4.1 Quantitative Formula Sheet

Topic	Formulas & shortcuts
Percentage	Increase by a% then decrease by a% → net change = $-a^2/100$ %. Successive change: $a + b + ab/100$.
Profit & Loss	$SP = CP(100+P)/100$. Two items sold at same price, one at +a%, one at −a% → always a loss of $a^2/100$ %.
Simple Interest	$SI = PRT/100$
Compound Interest	$A = P(1 + R/100)^T$. CI − SI for 2 years = $PR^2/100^2$.
Ratio & Proportion	If $a:b = c:d$ then $ad = bc$. Divide N in ratio $a:b \rightarrow Na/(a+b), Nb/(a+b)$.
Average	$Avg = Sum/n$. New avg after adding x = $(n \cdot A + x)/(n+1)$. First n naturals avg = $(n+1)/2$.
Time & Work	A in a days, B in b days → together $ab/(a+b)$ days. Efficiency $\propto 1/time$.
Pipes & Cisterns	Same as time & work; outlet pipe is negative work.
Time, Speed, Distance	$D = S \times T$. Avg speed for equal distances = $2xy/(x+y)$. km/h → m/s: $\times 5/18$.
Trains	Same direction: relative speed = difference. Opposite: sum. Crossing a pole: length/speed. Crossing a platform: $(L_{train} + L_{platform})/speed$.
Boats & Streams	Downstream = $u+v$, Upstream = $u-v$, $u = (D+U)/2$, $v = (D-U)/2$.
Permutation & Combination	$nPr = n!/(n-r)!$, $nCr = n!/(r!(n-r)!)$, $nCr = nC(n-r)$.
Probability	$P = \text{favourable/total}$. $P(A \text{ or } B) = P(A)+P(B)-P(A \text{ and } B)$.
HCF & LCM	$HCF \times LCM = \text{product of the two numbers}$.
Mensuration	Circle area πr^2 , circumference $2\pi r$. Cylinder vol $\pi r^2 h$, CSA $2\pi rh$. Sphere vol $4/3\pi r^3$, SA $4\pi r^2$. Cone vol $1/3\pi r^2 h$, CSA πrl .
Number System	Divisibility: 3 → digit sum; 4 → last two digits; 8 → last three; 9 → digit sum; 11 → alternating sum.

4.2 Logical & Critical Reasoning Types

Type	How to crack it fast
Number/Letter Series	Check differences, then second differences, then ratios, then squares/cubes $\pm k$. Letters: convert to 1–26 numbers.
Coding-Decoding	Find positional shift (+1, −2, reverse). Test on the shortest word first.
Blood Relations	Draw a tree. Male = square, Female = circle, generation = row.
Seating Arrangement	Fix the most constrained person first. Circular facing-centre: left = clockwise.
Syllogisms	Venn diagrams. "Some A are B" never gives "Some A are not B" as definite. Only conclusions true in every valid diagram follow.
Statement & Assumption	An assumption must be necessary for the statement to make sense, not merely consistent with it.
Cause & Effect	Effect follows cause in time; correlation alone is not causation.
Data Sufficiency	Never solve — only decide if it <i>can</i> be solved. Evaluate statements independently first.
Data Interpretation	Read axis labels and units before the question. Approximate aggressively.
Abstract Reasoning	Scan in this order: rotation → reflection → count of elements → shading → size → position.

4.3 English Ability Quick Notes

- **Subject-verb agreement:** the verb follows the subject, not the nearest noun. *The box of pens **is** on the table.*
- **Tense consistency:** reported speech shifts one tense back.
- **Common error traps:** its/it's, affect/effect, principal/principle, fewer/less, between/among, lie/lay.
- **Articles:** "a" before consonant *sound* (a university), "an" before vowel *sound* (an hour).
- **Reading comprehension:** read the questions first, then scan. Answer only from the passage, never from outside knowledge.
- **Para jumbles:** find the opening sentence (introduces a noun without a pronoun) and mandatory pairs (pronoun references).

5. Round 2 — Pseudocode (Highest-Weight Topic)

This is the single most valuable topic in the Technical Assessment: roughly 10–15 of 45 questions. Almost every question asks "What will be the output of the following pseudocode?"

5.1 Accenture Pseudocode Syntax Rules

Construct	Meaning
Integer x, y	Declaration. Integer division truncates toward zero: $7/2 = 3$.
Set x = 5	Assignment.
Print x	Output. Multiple prints appear space-separated on one line unless stated.
if ... End if	Conditional. Indentation shows nesting — trust the End if, not the indentation .
do{ ... } while(cond)	Executes the body at least once , then checks.
while(cond) ... End while	Checks first; may execute zero times.
for(i=0; i<n; i++)	Standard loop.
Integer arr[5]	Array, usually 0-indexed. Watch the question — some sets use 1-indexed.
Function f(Integer n)	Function; may be recursive.
&	Bitwise AND (not logical AND)
	Bitwise OR
^	Bitwise XOR — NOT exponentiation. This is the #1 trap.
~	Bitwise NOT: $\sim n = -(n+1)$
<< , >>	Left shift ($\times 2$ per shift), right shift ($\div 2$ per shift, floor)
%	Modulo (remainder)
&& , , !	Logical AND, OR, NOT

THE trap: $a = c^a$ means **c XOR a**, not c raised to a. In Accenture pseudocode, ^ is always bitwise XOR. Candidates lose 3–5 marks per paper on this alone.

5.2 Bitwise Cheat Table (memorise)

Operation	Rule	Example
AND &	1 only if both bits are 1	$11 \& 5 = 1011 \& 0101 = 0001 = \mathbf{1}$
OR	1 if either bit is 1	$12 5 = 1100 0101 = 1101 = \mathbf{13}$
XOR ^	1 only if bits differ	$7 \wedge 5 = 0111 \wedge 0101 = 0010 = \mathbf{2}$
NOT ~	$\sim n = -(n+1)$	$\sim 5 = \mathbf{-6}$
Left shift	$n \ll k = n \times 2^k$	$5 \ll 2 = \mathbf{20}$
Right shift	$n \gg k = \text{floor}(n / 2^k)$	$20 \gg 2 = \mathbf{5}$, $7 \gg 1 = \mathbf{3}$

XOR identities you will need: $a \wedge a = 0$ • $a \wedge 0 = a$ • XOR is commutative and associative • $a \wedge b \wedge a = b$.

Binary of 0–15: 0000 0001 0010 0011 0100 0101 0110 0111 1000 1001 1010 1011 1100 1101 1110 1111

5.3 The Tracing Method (use this on every question)

1. **Draw a variable table** on your rough sheet with one column per variable.
2. **Write the initial values** in row 0.
3. **Execute one line at a time.** Never skip an assignment mentally.
4. **Match every if with its End if** before evaluating anything. Number the lines if the nesting is deep.
5. **Mark every Print** in a separate "OUTPUT" column as it happens.
6. **For loops, write one row per iteration** — including the final iteration that fails the condition.

5.4 Solved PYQ 1 — do-while trace (from the SRMIST deck)

```

Integer x, y
Set x = 15, y = 12
y = x - 1
do{
    Print x
    x = y + (x - 2)
}
while(x < 40)
end do while

```

Options: (a) 14 26 (b) 15 27 39 (c) 27 39 (d) 15

Trace: $y = 15 - 1 = 14$ (y is overwritten; the initial 12 is a decoy).

Iteration	Print	$x = y + (x - 2)$	$x < 40 ?$
1	15	$14 + 13 = 27$	$27 < 40 \rightarrow \text{true}$
2	27	$14 + 25 = 39$	$39 < 40 \rightarrow \text{true}$
3	39	$14 + 37 = 51$	$51 < 40 \rightarrow \text{false, exit}$

Answer: (b) 15 27 39
 Key insights: (1) $y = x - 1$ discards the initial $y = 12$. (2) do-while prints *before* checking, so 15 appears. (3) The loop exits *after* printing 39 — 51 is never printed.

5.5 Solved PYQ 2 — nested if with bitwise (from the SRMIST deck)

```

1. Integer a,b,c
2. Set a=5, b=2, c=7
3. if((1-c-b)>(b-a))
4.     if(2>a)
5.         if((b&c)<(5-b))
6.             a=c^a
7.         End if
8.         b=9+b
9.     Else
10.        c=11&a
11.    End if
12.    b=(c+3)^b
13. End if
14. c=(b+c)+b
15. Print a+b+c

```

Options: (a) 16 (b) 18 (c) 10 (d) 19

Step 1 — map the blocks. Line 3's `if` closes at line 13. So lines 4-12 are its body. Line 4's `if` has its `Else` at line 9 and closes at line 11.

Step 2 — evaluate the outer condition.

$$\text{LHS} = 1 - c - b = 1 - 7 - 2 = -8$$

$$\text{RHS} = b - a = 2 - 5 = -3$$

Is $-8 > -3$? **No.** The condition is false.

Step 3 — skip the entire body. Lines 4 through 12 never execute. There is no outer `Else`, so control jumps to line 14. Values remain $a=5, b=2, c=7$.

Step 4 — line 14. $c = (b + c) + b = (2 + 7) + 2 = 11$

Step 5 — line 15. $a + b + c = 5 + 2 + 11 = 18$

Answer: (b) 18
 This question is designed so that candidates who rush into the bitwise lines (6, 10, 12) waste 90 seconds computing values that are never used. **Always evaluate the outermost condition first.**

5.6 Pseudocode Practice Bank — 25 Solved Questions

Q1. Output?

```

Integer a, b
Set a = 4, b = 5
a = a ^ b
b = a ^ b
a = a ^ b
Print a, b

```

5 4. Classic XOR swap. $a=4^5=1; b=1^5=4; a=1^4=5$.

Q2. Output?

```
Integer i, sum
Set sum = 0
for(i = 1; i <= 5; i++)
{
    if(i % 2 == 0)
        sum = sum + i
    End if
}
Print sum
```

6. Even values 2 and 4 $\rightarrow 2 + 4 = 6$.

Q3. Output?

```
Integer x
Set x = 10
while(x > 0)
{
    x = x - 3
}
Print x
```

-2. $10 \rightarrow 7 \rightarrow 4 \rightarrow 1 \rightarrow -2$, then $-2 > 0$ is false.

Q4. Output?

```
Integer a, b, c
Set a = 12, b = 10, c = 0
c = a & b
Print c
```

8. $12 = 1100, 10 = 1010, \text{AND} = 1000 = 8$.

Q5. Output?

```
Integer n, f
Set n = 5, f = 1
while(n > 1)
{
    f = f * n
    n = n - 1
}
Print f
```

120. Factorial of 5.

Q6. Output?

```
Integer arr[5] = {3, 7, 1, 9, 4}
Integer i, max
Set max = arr[0]
for(i = 1; i < 5; i++)
{
    if(arr[i] > max)
        max = arr[i]
    End if
}
Print max
```

9.

Q7. Output?

```
Integer a
Set a = 7
Print a >> 1
Print a << 2
```

3 28. $7 >> 1 = \text{floor}(7/2) = 3; 7 << 2 = 7 \times 4 = 28$.

Q8. Output?

```
Function fun(Integer n)
{
    if(n <= 1)
        return 1
    End if
    return n * fun(n - 2)
}
Print fun(7)
```

105. $7 \times 5 \times 3 \times \text{fun}(1) = 7 \times 5 \times 3 \times 1 = 105$.

Q9. Output?

```
Integer i, j, count
Set count = 0
for(i = 1; i <= 3; i++)
{
    for(j = i; j <= 3; j++)
    {
        count = count + 1
    }
}
Print count
```

6. Inner runs $3 + 2 + 1$ times.

Q10. Output?

```
Integer a, b
Set a = 15, b = 4
Print a / b
Print a % b
```

3 3. Integer division truncates: $15/4 = 3$; remainder 3.

Q11. Output?

```
Integer x, y
Set x = 3, y = 5
if(x > 2 && y < 4)
    Print "A"
Else if(x > 2 || y < 4)
    Print "B"
Else
    Print "C"
End if
```

B. First condition: $\text{true} \ \&\& \ \text{false} = \text{false}$. Second: $\text{true} \ || \ \text{false} = \text{true}$.

Q12. Output?

```
Integer n, rev, d
Set n = 342, rev = 0
while(n > 0)
{
    d = n % 10
    rev = rev * 10 + d
    n = n / 10
}
Print rev
```

243. Standard digit-reversal idiom — learn it, it appears in almost every paper.

Q13. Output?

```
Integer a
Set a = 5
Print ~a
```

-6. $\sim n = -(n+1)$.

Q14. Output?

```
Integer arr[6] = {2, 4, 6, 8, 10, 12}
Integer i, s
Set s = 0
for(i = 0; i < 6; i = i + 2)
{
    s = s + arr[i]
}
Print s
```

18. Indices 0, 2, 4 $\rightarrow 2 + 6 + 10 = 18$.

Q15. Output?

```
Integer i
Set i = 0
do{
    Print i
    i = i + 1
}
while(i < 0)
```

0. do-while always runs once even though the condition is false from the start.

Q16. Output?

```
Integer a, b, c
Set a = 8, b = 3
c = a ^ b
Print c
```

11. $1000 \wedge 0011 = 1011 = 11$.

Q17. Output?

```
Function f(Integer n)
{
    if(n == 0) return 0 End if
    if(n == 1) return 1 End if
    return f(n-1) + f(n-2)
}
Print f(6)
```

8. Fibonacci: 0,1,1,2,3,5,8.

Q18. Output?

```
Integer i, n, s
Set n = 4, s = 0
for(i = 1; i <= n; i++)
{
    s = s + i * i
}
Print s
```

30. $1+4+9+16$. Shortcut: $n(n+1)(2n+1)/6 = 4 \cdot 5 \cdot 9/6 = 30$.

Q19. Output?

```
Integer a, b
Set a = 10, b = 20
if(a = 5)
    Print "equal"
Else
    Print "not equal"
End if
Print a
```

equal, then 5. Trap: single `=` is **assignment**, not comparison. `a` becomes 5, and 5 is truthy (non-zero). If the paper treats `=` as comparison, the answer is "not equal" and 10 — check the option list to infer the convention.

Q20. Output?

```
Integer arr[5] = {5, 2, 8, 1, 9}
Integer i, j, t
for(i = 0; i < 4; i++)
{
    for(j = 0; j < 4 - i; j++)
    {
        if(arr[j] > arr[j+1])
        {
            t = arr[j]
            arr[j] = arr[j+1]
            arr[j+1] = t
        }
    }
}
Print arr[0], arr[4]
```

1 9. Bubble sort ascending $\rightarrow \{1,2,5,8,9\}$.

Q21. Output?

```

Integer n, c
Set n = 29, c = 0
Integer i
for(i = 1; i <= n; i++)
{
    if(n % i == 0)
        c = c + 1
    End if
}
Print c

```

2. 29 is prime, so divisors are 1 and 29.

Q22. Output?

```

Integer a, b, c
Set a = 2, b = 3, c = 4
a = a + b
b = a - b
c = a * b - c
Print a, b, c

```

5 2 6. $a=5$; $b=5-3=2$; $c=5 \times 2 - 4=6$.

Q23. Output?

```

String s = "ACCENTURE"
Integer i, c
Set c = 0
for(i = 0; i < length(s); i++)
{
    if(s[i] == 'C' || s[i] == 'E')
        c = c + 1
    End if
}
Print c

```

4. ACCENTURE = A,C,C,E,N,T,U,R,E. C appears at indices 1 and 2; E appears at indices 3 and 8. $2 + 2 = 4$. *Always write the string out with indices — counting from memory is how people get 5 here.*

Q24. Output?

```

Integer x
Set x = 1
Integer i
for(i = 1; i <= 3; i++)
{
    x = x * 2
    if(x > 4)
        break
    End if
}
Print x, i

```

8 3. $i=1 \rightarrow x=2$; $i=2 \rightarrow x=4$ ($4 > 4$ false); $i=3 \rightarrow x=8$, break with i still 3.

Q25. Output?

```

Integer a[4] = {1, 2, 3, 4}
Integer i, s
Set s = 0
for(i = 0; i < 4; i++)
{
    if(i % 2 == 0)
        s = s + a[i]
    Else
        s = s - a[i]
    End if
}
Print s

```

-2. $+1 -2 +3 -4 = -2$.

Pseudocode exam tips: (1) If a trace exceeds ~8 iterations, look for a closed-form pattern instead. (2) Eliminate options by parity or magnitude before tracing fully. (3) Any variable assigned but never used is a decoy — note it and move on. (4) Budget 60–70 seconds per pseudocode question; mark-and-return if you exceed it.

6. Round 2 — Common Applications & MS Office

Usually 5–8 easy marks. Do not skip this — it is the cheapest section in the paper.

6.1 MS Word Cheat Notes

Feature	Key fact
Default extension	.docx (Word 2007+); template .dotx; macro-enabled .docm
Mail Merge	Main document + Data source → merged letters/labels/emails. Mailings tab.
Header & Footer	Insert tab; page numbers, date, file path go here
Track Changes	Review tab, Ctrl+Shift+E — records insertions/deletions for review
Section Break	Lets different pages have different orientation/headers
Styles	Basis of automatic Table of Contents; heading styles drive TOC levels
Ctrl shortcuts	Ctrl+B bold, I italic, U underline, S save, P print, F find, H replace, Z undo, Y redo, K hyperlink, Enter page break

6.2 MS Excel Cheat Notes (highest-yield)

Function	Purpose / syntax
SUM / AVERAGE	=SUM(A1:A10)
COUNT / COUNTA / COUNTBLANK	COUNT = numbers only; COUNTA = non-empty cells; COUNTBLANK = empty
IF	=IF(condition, value_if_true, value_if_false)
VLOOKUP	=VLOOKUP(lookup, table, col_index, [range_lookup]). FALSE = exact match. Searches leftmost column only.
HLOOKUP	Same but horizontal (searches the first row)
INDEX + MATCH	More flexible than VLOOKUP; can look left
CONCATENATE / &	Join text: =A1&" "&B1
LEN / LEFT / RIGHT / MID / TRIM	Text functions; TRIM removes extra spaces
SUMIF / COUNTIF / AVERAGEIF	Conditional aggregates; ...IFS for multiple criteria
Absolute vs relative	\$A\$1 absolute, A1 relative, \$A1 / A\$1 mixed. F4 toggles.
Pivot Table	Summarises large data by rows/columns/values; Insert tab
Conditional Formatting	Formats cells based on rules/values
Freeze Panes	View tab — locks headers while scrolling
Extensions	.xlsx normal, .xlsm macro-enabled, .xlsb binary, .csv plain text
Error values	#DIV/0! divide by zero • #N/A not found • #REF! invalid reference • #VALUE! wrong type • #NAME? unrecognised name • #NUM! invalid number
Shortcuts	Ctrl+; today's date • Ctrl+Shift+L filter • Alt+= autosum • F2 edit cell • Ctrl+Arrow jump to edge

6.3 MS PowerPoint & Outlook

- **PowerPoint:** .pptx (template .potx, show .ppsx). F5 = start slideshow from beginning; Shift+F5 = from current slide. **Slide Master** controls the design of all slides. Transition = between slides; Animation = within a slide.
- **Outlook:** CC visible to all; **BCC hidden**. .pst is the local data file. Rules automate inbox sorting. Calendar invites use the iCalendar format.

6.4 Practice MCQs

Question	Answer
Which Excel function returns the number of non-empty cells?	COUNTA
Which key makes a cell reference absolute?	F4
What does #REF! indicate?	A reference to a deleted/invalid cell
VLOOKUP searches which column of the table array?	The leftmost (first) column
Which feature merges a letter with a recipient list in Word?	Mail Merge
Shortcut to start a PowerPoint slideshow from the current slide?	Shift + F5
Which recipient field hides addresses from other recipients?	BCC
Which Excel feature locks header rows while scrolling?	Freeze Panes

Which file extension stores an Excel macro-enabled workbook?	.xlsm
Which Word feature records edits for later acceptance/rejection?	Track Changes

7. Round 2 — Object-Oriented Programming (OOPs)

7.1 The Four Pillars

Pillar	Definition	Real example
Encapsulation	Binding data and the methods that operate on it into one unit; hiding internal state behind access modifiers	private fields + public getters/setters
Abstraction	Exposing only essential features and hiding implementation detail	abstract class / interface; driving a car without knowing the engine
Inheritance	A class acquires properties and behaviour of another class (IS-A relationship)	class Dog extends Animal
Polymorphism	One interface, many forms	Overloading (compile-time), Overriding (runtime)

7.2 Overloading vs Overriding — the most-asked comparison

Aspect	Overloading	Overriding
Also called	Compile-time / static polymorphism	Runtime / dynamic polymorphism
Where	Same class (or subclass)	Parent and child class
Signature	Must differ (number/type/order of parameters)	Must be identical
Return type	Can differ	Same or covariant
Access modifier	Any	Cannot be more restrictive than parent
Binding	Early binding	Late binding
static / final / private methods	Can be overloaded	Cannot be overridden

7.3 Key Concepts Table

Concept	What you must remember
Class vs Object	Class is the blueprint; object is the instance occupying memory
Constructor	Same name as class, no return type, called automatically on object creation. Can be overloaded, never overridden.
Destructor / finalize	Releases resources; in Java handled by the Garbage Collector
Abstract class	Cannot be instantiated; may have both abstract and concrete methods; supports constructors and instance variables
Interface	100% abstract contract (pre-Java 8); supports multiple inheritance; fields are public static final
Abstract class vs Interface	Use abstract class for "IS-A with shared code"; interface for "CAN-DO capability"
Access modifiers	private < default(package) < protected < public
static	Belongs to the class, not the instance; shared across all objects; cannot access non-static members directly
final	final variable = constant; final method = cannot override; final class = cannot inherit
this vs super	this = current object; super = immediate parent
Types of inheritance	Single, Multilevel, Hierarchical, Multiple, Hybrid. Java does not support multiple inheritance of classes (Diamond Problem) — it uses interfaces instead.
Association / Aggregation / Composition	Association = general link. Aggregation = HAS-A, weak, parts survive the whole. Composition = HAS-A, strong, parts die with the whole.
Coupling & Cohesion	Good design = low coupling, high cohesion
Exception handling	try-catch-finally-throw-throws. <code>finally</code> always executes (except <code>System.exit</code>). Checked = compile-time, Unchecked = runtime.
Garbage Collection	Automatic reclamation of unreachable objects; <code>System.gc()</code> only <i>requests</i> collection
SOLID	Single responsibility, Open/closed, Liskov substitution, Interface segregation, Dependency inversion

7.4 Practice MCQs with Answers

Question	Answer
Which OOP feature allows the same function name with different parameter lists?	Method overloading
Which principle hides internal state from outside access?	Encapsulation
Can a constructor be overridden?	No (it can only be overloaded)

Why does Java avoid multiple inheritance of classes?	The Diamond Problem / ambiguity
Which keyword prevents a class from being inherited?	final
An abstract class with all abstract methods is effectively a...?	Interface
Which block always executes regardless of exception?	finally
Which is a runtime (unchecked) exception?	NullPointerException / ArrayIndexOutOfBoundsException
Which relationship is strongest: Association, Aggregation, Composition?	Composition
Can a static method access a non-static variable directly?	No
Runtime polymorphism in Java is achieved through?	Method overriding (dynamic dispatch)
Default access modifier in Java when none is specified?	package-private (default)

8. Round 2 — DBMS & SQL

8.1 Core DBMS Concepts

Concept	Must-know
DBMS vs RDBMS	RDBMS stores data in related tables and enforces relationships/constraints
Keys	Primary unique + not null, one per table. Candidate eligible to be primary. Super any uniquely-identifying set. Foreign references a primary key in another table. Composite made of 2+ columns. Unique allows one NULL.
Normalization	1NF atomic values, no repeating groups. 2NF 1NF + no partial dependency on part of a composite key. 3NF 2NF + no transitive dependency. BCNF every determinant is a candidate key.
Denormalization	Deliberately adding redundancy to speed up reads
ACID	A tomicity (all or nothing), C onsistency, I solation, D urability
Index	Speeds up SELECT, slows down INSERT/UPDATE/DELETE. Clustered = physical order (one per table); Non-clustered = separate structure (many allowed).
View	A virtual table based on a query. Materialized view stores the result physically.
Stored Procedure vs Function	Procedure may return zero or many values and can modify data; function must return a value and is usable inside a query
Trigger	Code executed automatically on INSERT/UPDATE/DELETE
Transaction control	COMMIT, ROLLBACK, SAVEPOINT

8.2 SQL Command Categories

Type	Commands	Notes
DDL	CREATE, ALTER, DROP, TRUNCATE, RENAME	Auto-committed, structural
DML	INSERT, UPDATE, DELETE	Can be rolled back
DQL	SELECT	Retrieval
DCL	GRANT, REVOKE	Permissions
TCL	COMMIT, ROLLBACK, SAVEPOINT	Transactions

Classic MCQ: DELETE vs TRUNCATE vs DROP.
DELETE — DML, row-by-row, WHERE allowed, can rollback, fires triggers, keeps structure.
TRUNCATE — DDL, removes all rows fast, no WHERE, cannot rollback (in most engines), resets identity, keeps structure.
DROP — DDL, removes the table itself along with data, indexes and constraints.

8.3 JOINS (guaranteed question)

Join	Returns
INNER JOIN	Only matching rows from both tables
LEFT (OUTER) JOIN	All rows from left + matches from right; NULLs where no match
RIGHT (OUTER) JOIN	All rows from right + matches from left
FULL OUTER JOIN	All rows from both, NULL-padded
CROSS JOIN	Cartesian product (m × n rows)
SELF JOIN	A table joined to itself using aliases (e.g., employee → manager)

8.4 Query Execution Order (memorise this — it explains most SQL mistakes)

FROM → JOIN → WHERE → GROUP BY → HAVING → SELECT → DISTINCT → ORDER BY → LIMIT

Why it matters: because WHERE runs before GROUP BY, you cannot use an aggregate in WHERE — you must use HAVING. And because SELECT runs after GROUP BY, a column alias defined in SELECT cannot be used in WHERE (but can be used in ORDER BY).

8.5 Functions You Must Know

Category	Functions
Aggregate	COUNT(), SUM(), AVG(), MIN(), MAX() — COUNT(*) counts NULLs, COUNT(col) does not
String	UPPER, LOWER, LENGTH/LEN, SUBSTRING, CONCAT, TRIM, REPLACE
Date	NOW(), CURDATE(), YEAR(), MONTH(), DATEDIFF()

NULL handling	IS NULL, IS NOT NULL, COALESCE(), IFNULL()/ISNULL(), NULLIF(). NULL = NULL is never TRUE.
Window	ROW_NUMBER(), RANK(), DENSE_RANK() OVER (PARTITION BY ... ORDER BY ...)
Operators	IN, BETWEEN, LIKE ('%' any chars, '_' one char), EXISTS, ANY, ALL

8.6 Practice MCQs with Answers

Question	Answer
Which clause filters groups after aggregation?	HAVING
Which key can accept exactly one NULL value?	UNIQUE key
Which command removes all rows but cannot normally be rolled back?	TRUNCATE
Which join produces the Cartesian product?	CROSS JOIN
Which normal form removes transitive dependency?	3NF
How many clustered indexes can a table have?	One
COUNT(*) vs COUNT(column) difference?	COUNT(*) includes rows with NULLs
Which is a DCL command?	GRANT
Query to find the second-highest salary?	SELECT MAX(sal) FROM emp WHERE sal < (SELECT MAX(sal) FROM emp)
Which SQL keyword eliminates duplicate rows?	DISTINCT
Wildcard matching exactly one character in LIKE?	Underscore _
Property guaranteeing a transaction is all-or-nothing?	Atomicity

8.7 Ten SQL Queries to Have Memorised

```
-- 1. Second highest salary
SELECT MAX(salary) FROM employees
WHERE salary < (SELECT MAX(salary) FROM employees);

-- 2. Nth highest salary (window function)
SELECT DISTINCT salary FROM (
  SELECT salary, DENSE_RANK() OVER (ORDER BY salary DESC) rk
  FROM employees) t
WHERE rk = 3;

-- 3. Duplicate rows
SELECT email, COUNT(*) FROM users
GROUP BY email HAVING COUNT(*) > 1;

-- 4. Delete duplicates keeping the lowest id
DELETE FROM users WHERE id NOT IN
(SELECT MIN(id) FROM users GROUP BY email);

-- 5. Employees earning more than their department average
SELECT e.name FROM employees e
WHERE e.salary > (SELECT AVG(salary) FROM employees
  WHERE dept_id = e.dept_id);

-- 6. Department-wise highest paid
SELECT dept_id, MAX(salary) FROM employees GROUP BY dept_id;

-- 7. Employees with no manager (self join)
SELECT e.name FROM employees e
LEFT JOIN employees m ON e.manager_id = m.emp_id
WHERE m.emp_id IS NULL;

-- 8. Customers who never ordered
SELECT c.name FROM customers c
LEFT JOIN orders o ON c.id = o.customer_id
WHERE o.id IS NULL;

-- 9. Top 5 by revenue
SELECT product, SUM(qty*price) rev FROM sales
GROUP BY product ORDER BY rev DESC LIMIT 5;

-- 10. Month-wise count for a year
SELECT MONTH(order_date) m, COUNT(*) FROM orders
WHERE YEAR(order_date) = 2025 GROUP BY MONTH(order_date);
```

9. Round 2 — Operating Systems, CPU & Memory

9.1 OS Core Concepts

Concept	Must-know
Process vs Thread	Process = independent, own address space, heavy context switch. Thread = lightweight, shares code/data/heap of its process, has its own stack, PC and registers.
Process states	New → Ready → Running → Waiting/Blocked → Terminated
PCB	Process Control Block: PID, state, program counter, registers, memory limits, open files
Context switch	Saving one process's state and loading another's — pure overhead
Scheduling algorithms	FCFS (convoy effect), SJF (optimal avg wait, needs burst prediction), SRTF (preemptive SJF), Priority (starvation → fix with aging), Round Robin (time quantum, best for time-sharing), Multilevel Queue
Preemptive vs Non-preemptive	Preemptive: OS can forcibly take the CPU (RR, SRTF). Non-preemptive: process runs to completion/block (FCFS, SJF).
Deadlock — 4 necessary conditions	Mutual exclusion, Hold and wait, No preemption, Circular wait — all four must hold
Deadlock handling	Prevention (break a condition), Avoidance (Banker's Algorithm), Detection & Recovery, Ignorance (Ostrich)
Starvation vs Deadlock	Starvation: process waits indefinitely but others progress. Deadlock: nobody progresses.
Critical section	Needs Mutual exclusion + Progress + Bounded waiting. Solved with semaphores, mutex, monitors.
Semaphore vs Mutex	Mutex = binary lock, ownership enforced. Semaphore = counter, signalling, no ownership. Counting semaphore allows N concurrent.
IPC	Pipes, message queues, shared memory (fastest), sockets, signals

9.2 Memory Management

Concept	Must-know
Paging	Fixed-size blocks: pages (logical) mapped to frames (physical). Causes internal fragmentation .
Segmentation	Variable-size logical divisions (code, stack, data). Causes external fragmentation .
Virtual memory	Illusion of more memory than physical RAM, using disk as backing store. Enabled by demand paging.
Page fault	Referenced page not in memory → OS fetches it from disk
Page replacement	FIFO (suffers Belady's anomaly), Optimal (theoretical best), LRU (practical), LFU, Clock
Thrashing	Excessive paging; CPU spends more time swapping than executing. Fix: reduce degree of multiprogramming, working-set model.
TLB	Translation Lookaside Buffer — cache for page table entries
Fragmentation	Internal = unused space inside an allocated block. External = free space scattered in unusable small pieces. Fix: compaction / paging.
Allocation strategies	First fit (fast), Best fit (least waste, slow), Worst fit

9.3 CPU & Hardware

Concept	Must-know
CPU components	ALU (arithmetic/logic), CU (control unit), Registers, Cache
Memory hierarchy	Registers → L1 → L2 → L3 cache → RAM → SSD/HDD → Tape. Faster = smaller = costlier.
RAM vs ROM	RAM volatile read/write; ROM non-volatile, holds firmware/BIOS
SRAM vs DRAM	SRAM: flip-flops, fast, expensive, used for cache. DRAM: capacitors, needs refresh, used for main memory.
Cache mapping	Direct-mapped, Fully associative, Set associative
Cache hit/miss	Hit ratio = hits / total accesses. Locality: temporal (recently used again) and spatial (nearby addresses).
Instruction cycle	Fetch → Decode → Execute → (Memory) → Write-back
Pipelining	Overlapping instruction stages to raise throughput; hazards: structural, data, control
RISC vs CISC	RISC: few simple fixed-length instructions, more registers, pipeline-friendly (ARM). CISC: many complex variable-length instructions (x86).
Buses	Address bus (where), Data bus (what), Control bus (how)
Units	1 byte = 8 bits; 1 KB = 1024 B; 1 MB = 1024 KB; 1 GB = 1024 MB
32-bit vs 64-bit	32-bit addresses max 4 GB RAM (2^{32}); 64-bit vastly more

9.4 Practice MCQs with Answers

Question	Answer
Which scheduling algorithm gives the minimum average waiting time?	SJF (Shortest Job First)
Which page replacement algorithm suffers Belady's anomaly?	FIFO
Which is NOT a necessary condition for deadlock: mutual exclusion, hold and wait, preemption, circular wait?	Preemption (the condition is <i>No</i> preemption)
Paging causes which type of fragmentation?	Internal
Which algorithm is used for deadlock avoidance?	Banker's Algorithm
Threads of the same process share which of these: stack, registers, heap, program counter?	Heap (code and data too); stack/registers/PC are private
What is thrashing?	Excessive page faults causing CPU to spend most time swapping
Which memory is directly accessed by the CPU fastest?	Registers
TLB is a cache for?	Page table entries (address translations)
Which scheduling is best suited for time-sharing systems?	Round Robin
Which memory type requires periodic refreshing?	DRAM
Fix for starvation in priority scheduling?	Aging

10. Round 2 — Networking, Security, Cloud, Wi-Fi & DevOps

10.1 Networking Fundamentals

OSI Layer	Name	Function & protocols
7	Application	HTTP, HTTPS, FTP, SMTP, DNS, DHCP, SNMP, Telnet, SSH
6	Presentation	Encryption, compression, translation (SSL/TLS, JPEG, ASCII)
5	Session	Establish/manage/terminate sessions (NetBIOS, RPC)
4	Transport	End-to-end delivery, segmentation (TCP, UDP)
3	Network	Logical addressing and routing (IP, ICMP, ARP*, routers)
2	Data Link	MAC addressing, framing, error detection (switches, bridges)
1	Physical	Bits on the wire (cables, hubs, repeaters)

*ARP is usually placed at Layer 2/3 boundary. TCP/IP model has 4 layers: Application, Transport, Internet, Network Access.

TCP vs UDP

Aspect	TCP	UDP
Connection	Connection-oriented (3-way handshake: SYN, SYN-ACK, ACK)	Connectionless
Reliability	Guaranteed, acknowledged, retransmits	Best-effort, no guarantee
Ordering	Preserved	Not preserved
Speed / Overhead	Slower, 20-byte header	Faster, 8-byte header
Used by	HTTP, FTP, SMTP, SSH	DNS, DHCP, VoIP, streaming, online games

Port Numbers (memorise)

Port	Service	Port	Service
20/21	FTP (data/control)	80	HTTP
22	SSH / SFTP	110	POP3
23	Telnet	143	IMAP
25	SMTP	443	HTTPS
53	DNS	3306	MySQL
67/68	DHCP	3389	RDP

IP Addressing

- **IPv4** 32-bit, dotted decimal. **IPv6** 128-bit, hexadecimal, colon-separated.
- Classes: A (1–126), B (128–191), C (192–223), D multicast (224–239), E reserved (240–255). **127.x.x.x = loopback.**
- Private ranges: 10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16
- **NAT** maps private addresses to a public one. **DHCP** assigns IPs automatically. **DNS** resolves names to IPs.
- Subnet mask separates network and host portions. /24 = 255.255.255.0 = 256 addresses, 254 usable.

Devices

Hub (L1, broadcasts to all) • Switch (L2, MAC-based forwarding) • Router (L3, routes between networks) • Gateway (protocol translation) • Firewall (filters traffic by rules).

10.2 Information Security

Concept	Must-know
CIA Triad	Confidentiality, Integrity, Availability — the foundation of all security questions
AAA	Authentication (who you are), Authorization (what you may do), Accounting (what you did)
Symmetric encryption	One shared key; fast; AES, DES, 3DES, Blowfish
Asymmetric encryption	Public + private key pair; slower; RSA, ECC, Diffie-Hellman
Hashing	One-way, fixed length, irreversible; MD5 (broken), SHA-1 (weak), SHA-256 (safe). Used for integrity and password storage (with salt).
Digital signature	Sign with sender's private key, verify with sender's public key → authenticity + non-repudiation
SSL/TLS & HTTPS	TLS secures HTTP; uses asymmetric for key exchange then symmetric for the session
Firewall types	Packet filtering, Stateful inspection, Proxy, Next-gen (NGFW)
IDS vs IPS	IDS detects and alerts; IPS detects and blocks
VPN	Encrypted tunnel over a public network
Zero Trust	"Never trust, always verify" — no implicit trust based on network location

Attack Types (very commonly asked)

Attack	What it is	Defence
Phishing	Fraudulent message tricking users into revealing credentials	User awareness, email filtering, MFA
SQL Injection	Malicious SQL inserted through input fields	Parameterized queries / prepared statements, input validation
XSS	Malicious script injected into a web page viewed by others	Output encoding, CSP, sanitise input
CSRF	Forces an authenticated user to submit an unwanted request	Anti-CSRF tokens, SameSite cookies
DoS / DDoS	Flooding a service to make it unavailable	Rate limiting, CDN, scrubbing services
Man-in-the-Middle	Attacker intercepts communication between two parties	TLS, certificate pinning, avoid open Wi-Fi
Ransomware	Encrypts victim data and demands payment	Offline backups, patching, least privilege
Brute force / Credential stuffing	Guessing or reusing leaked passwords	MFA, lockout, strong password policy
Social engineering	Manipulating people rather than systems	Training, verification procedures
Zero-day	Exploiting an unknown, unpatched vulnerability	Defence in depth, behaviour-based detection

10.3 Wi-Fi Security

Standard	Security level	Notes
WEP	Broken — never use	RC4, static key, trivially cracked
WPA	Weak	TKIP, interim fix for WEP
WPA2	Good (still common)	AES-CCMP; vulnerable to KRACK on unpatched clients
WPA3	Current best	SAE (Simultaneous Authentication of Equals) replaces PSK handshake; forward secrecy; protects against offline dictionary attacks

- **WPA2/3-Personal** uses a pre-shared key (PSK); **Enterprise** uses 802.1X with a RADIUS server.
- **SSID hiding** and **MAC filtering** are weak, cosmetic measures — not real security.
- **Evil Twin** = rogue AP imitating a legitimate SSID. **Rogue AP** = unauthorised AP on your network. **Deauthentication attack** forces clients off to capture handshakes.
- **WPS** should be disabled — its 8-digit PIN is brute-forceable.
- On public Wi-Fi: use a VPN, ensure HTTPS, disable auto-connect and file sharing.

10.4 Cloud Computing

Model	You manage	Examples
IaaS	OS, runtime, apps, data (provider gives compute/storage/network)	AWS EC2, Azure VMs, Google Compute Engine
PaaS	Apps and data only	AWS Elastic Beanstalk, Azure App Service, Heroku
SaaS	Nothing — just use it	Gmail, Salesforce, Office 365

Memory hook: IaaS = you rent the kitchen. PaaS = you rent the kitchen with a chef's setup. SaaS = you order the meal.

Concept	Must-know
Deployment models	Public, Private, Hybrid, Community
Five essential characteristics	On-demand self-service, Broad network access, Resource pooling, Rapid elasticity, Measured service
Scaling	Vertical = bigger machine. Horizontal = more machines (preferred in cloud).
Virtualization	Hypervisor: Type 1 bare-metal (ESXi, Hyper-V), Type 2 hosted (VirtualBox)
VM vs Container	VM has its own guest OS (heavy, minutes to boot). Container shares the host kernel (light, seconds).
Shared Responsibility Model	Provider secures the cloud infrastructure ; customer secures their data, access and configuration
Key AWS services	EC2 compute, S3 object storage, RDS managed DB, Lambda serverless, VPC network, IAM access, CloudWatch monitoring
Key Azure services	Virtual Machines, Blob Storage, Azure SQL, Functions, Active Directory
Serverless	No server management, event-driven, pay per execution
CDN	Edge caching to reduce latency (CloudFront, Akamai)
SLA / RTO / RPO	SLA = promised uptime. RTO = how fast you recover. RPO = how much data you can afford to lose.

10.5 DevOps

Concept	Must-know
What DevOps is	Culture + practices uniting Development and Operations to deliver faster and more reliably
DevOps lifecycle	Plan → Code → Build → Test → Release → Deploy → Operate → Monitor (the infinity loop)
CI	Continuous Integration: merge and auto-test code frequently
CD	Continuous Delivery (ready to deploy, manual approval) vs Continuous Deployment (auto to production)
Version control	Git. Key commands: clone, add, commit, push, pull, fetch, branch, checkout, merge, rebase, stash, revert, reset
Merge vs Rebase	Merge preserves history with a merge commit; rebase rewrites for a linear history
CI/CD tools	Jenkins, GitLab CI, GitHub Actions, Azure DevOps, CircleCI, Bamboo
Containerization	Docker: Dockerfile → Image → Container. Commands: build, run, ps, images, pull, push, exec, stop
Orchestration	Kubernetes: Pod (smallest unit) → ReplicaSet → Deployment → Service → Ingress. Node, Cluster, kubectl.
Infrastructure as Code	Terraform (provisioning), Ansible / Chef / Puppet (configuration management)
Monitoring & logging	Prometheus + Grafana, ELK stack (Elasticsearch, Logstash, Kibana), Nagios, Splunk
Artifact repository	Nexus, JFrog Artifactory
Build tools	Maven, Gradle, Ant (Java), npm (JS)
Deployment strategies	Blue-Green (two identical environments, switch traffic), Canary (gradual rollout), Rolling update
DevSecOps	"Shift left" — build security into the pipeline from the start

10.6 Practice MCQs with Answers

Question	Answer
Which layer of the OSI model does a router operate at?	Layer 3 (Network)
Default port for HTTPS?	443
Which protocol assigns IP addresses dynamically?	DHCP
TCP three-way handshake sequence?	SYN → SYN-ACK → ACK
Which Wi-Fi standard uses SAE instead of PSK?	WPA3
Which cloud model gives the customer the least management responsibility?	SaaS
Which attack is prevented by using prepared statements?	SQL Injection
Which is a symmetric encryption algorithm?	AES
A digital signature is created using which key?	The sender's private key
Smallest deployable unit in Kubernetes?	Pod
Which tool is used for infrastructure provisioning as code?	Terraform
Difference between IDS and IPS?	IPS actively blocks; IDS only alerts
Which deployment strategy runs two identical production environments?	Blue-Green
Which protocol is connectionless?	UDP
What does the CIA triad stand for?	Confidentiality, Integrity, Availability
Which Git command combines fetch and merge?	git pull

11. Round 3 — Coding: DSA Patterns & Solved Problems

Q1 of the coding round is DSA/basic programming in Java, Python or C#. Accenture's DSA questions are rarely "hard" in the competitive-programming sense — they are usually medium-length simulation/array/string problems with a twist in the rules, similar to the deck's own examples. The skill tested is **careful reading** as much as algorithms.

11.1 Pattern Checklist (recognise before you code)

Signal in the question	Likely pattern
"contiguous subarray", "window of size k"	Sliding window
"pair that sums to", "sorted array"	Two pointers
"prefix sums", "range sum queries"	Prefix sum array
"count frequency", "anagram", "duplicate"	HashMap / frequency array
"every index i", transformation rules	Direct simulation loop (no fancy DS needed)
"digits of a number", "reverse", "palindrome"	Modulo/divide digit extraction
"maximum/minimum subject to constraint"	Greedy or DP — try greedy first, verify with an example
"parentheses", "nested", "undo"	Stack
"shortest path", "level by level"	BFS
"all combinations/permutations"	Backtracking / recursion

11.2 Solved: the exact deck DSA question ("Equivalent transform")

Problem (from deck): Given an array of integers `nums`, transform each element based on its 0-based index `i`: (1) subtract $(i \% 7) * 3$ from the element, (2) if the original element is divisible by 11, add $nums[i]/11$ to the modified value. Return the total sum after transformation. Example: input `{22, 5, 14}` → output `{24, 2, 8}` → sum 34.

Verify the example first (always do this):

$i=0$: $22 - (0\%7) \times 3 = 22 - 0 = 22$. 22 is divisible by 11 → $+22/11=+2 \rightarrow 24$. ✓
 $i=1$: $5 - (1\%7) \times 3 = 5 - 3 = 2$. 5 not divisible by 11 → stays 2. ✓
 $i=2$: $14 - (2\%7) \times 3 = 14 - 6 = 8$. 14 not divisible by 11 → stays 8. ✓
Sum = $24+2+8 = 34$. ✓ Matches — the rule is confirmed.

```
Python:
def equivalent_sum(nums):
    total = 0
    for i, val in enumerate(nums):
        modified = val - (i % 7) * 3
        if val % 11 == 0:
            modified += val // 11
        total += modified
    return total

print(equivalent_sum([22, 5, 14])) # 34
```

```
Java:
public static int equivalentSum(int[] nums) {
    int total = 0;
    for (int i = 0; i < nums.length; i++) {
        int modified = nums[i] - (i % 7) * 3;
        if (nums[i] % 11 == 0) {
            modified += nums[i] / 11;
        }
        total += modified;
    }
    return total;
}
```

Golden rule for every Accenture coding question: plug the given example into your logic *by hand* before writing code. If your hand-trace doesn't match the sample output, your understanding of the rules is wrong — fix that before touching the editor. This single habit prevents most "wrong answer" submissions.

11.3 Solved: the deck's "Equivalent Sum of prefixes" question

Problem: For integer `X` with digits `d1 d2 ... dk`, $EqSum(X) = \text{sum of every prefix number: } d1 + \text{int}(d1d2) + \dots + \text{int}(d1d2\dots dk)$. Example: $X=112 \rightarrow \text{prefixes } 1, 11, 112 \rightarrow EqSum = 1+11+112 = 124$.

```
Python:
def eq_sum(n):
    s = str(n)
    total = 0
    for i in range(1, len(s) + 1):
        total += int(s[:i])
    return total

print(eq_sum(112)) # 124
```

11.4 20 Practice Problems with Solutions (Python)

1. Reverse an integer without converting to string.

```
def reverse_num(n):
    rev = 0
    sign = -1 if n < 0 else 1
    n = abs(n)
    while n > 0:
        rev = rev * 10 + n % 10
        n //= 10
    return sign * rev
```

2. Check if a number is a palindrome.

```
def is_palindrome(n):
    return str(n) == str(n)[::-1]
```

3. Find the second largest element in an array.

```
def second_largest(arr):
    first = second = float('-inf')
    for x in arr:
        if x > first:
            second, first = first, x
        elif first > x > second:
            second = x
    return second
```

4. Check if two strings are anagrams.

```
def is_anagram(a, b):
    return sorted(a) == sorted(b)
```

5. Find all pairs in an array that sum to a target (two pointers, sorted).

```
def pair_sum(arr, target):
    arr.sort()
    l, r = 0, len(arr) - 1
    result = []
    while l < r:
        s = arr[l] + arr[r]
        if s == target:
            result.append((arr[l], arr[r]))
            l += 1; r -= 1
        elif s < target:
            l += 1
        else:
            r -= 1
    return result
```

6. Count frequency of each character in a string.

```
def char_freq(s):
    freq = {}
    for ch in s:
        freq[ch] = freq.get(ch, 0) + 1
    return freq
```

7. Find the missing number in an array of 1..n.

```
def missing_number(arr, n):
    expected = n * (n + 1) // 2
    return expected - sum(arr)
```

8. Check if a string has balanced parentheses.

```
def is_balanced(s):
    stack = []
    pairs = {'(': '(', ')': ')', '[': '[', ']': ']' }
    for ch in s:
        if ch in '([':
            stack.append(ch)
        elif ch in ')]':
            if not stack or stack.pop() != pairs[ch]:
                return False
    return not stack
```

9. Find the maximum sum of a contiguous subarray (Kadane's algorithm).

```
def max_subarray(arr):
    max_sum = cur = arr[0]
    for x in arr[1:]:
        cur = max(x, cur + x)
        max_sum = max(max_sum, cur)
    return max_sum
```

10. Rotate an array to the right by k steps.

```
def rotate(arr, k):
    n = len(arr)
    k %= n
    return arr[-k:] + arr[:-k]
```

11. Check if a number is prime.

```
def is_prime(n):
    if n < 2:
        return False
    for i in range(2, int(n**0.5) + 1):
        if n % i == 0:
            return False
    return True
```

12. Find all prime numbers up to n (Sieve of Eratosthenes).

```
def sieve(n):
    is_p = [True] * (n + 1)
    is_p[0] = is_p[1] = False
    for i in range(2, int(n**0.5) + 1):
        if is_p[i]:
            for j in range(i*i, n + 1, i):
                is_p[j] = False
    return [i for i, v in enumerate(is_p) if v]
```

13. Remove duplicates from an array while preserving order.

```
def remove_dupes(arr):
    seen = set()
    result = []
    for x in arr:
        if x not in seen:
            seen.add(x)
            result.append(x)
    return result
```

14. Find the first non-repeating character in a string.

```
def first_unique(s):
    freq = {}
    for ch in s:
        freq[ch] = freq.get(ch, 0) + 1
    for ch in s:
        if freq[ch] == 1:
            return ch
    return None
```

15. Merge two sorted arrays into one sorted array.

```
def merge_sorted(a, b):
    i = j = 0
    result = []
    while i < len(a) and j < len(b):
        if a[i] <= b[j]:
            result.append(a[i]); i += 1
        else:
            result.append(b[j]); j += 1
    result.extend(a[i:]); result.extend(b[j:])
    return result
```

16. Find the GCD and LCM of two numbers.

```
def gcd(a, b):
    while b:
        a, b = b, a % b
    return a

def lcm(a, b):
    return a * b // gcd(a, b)
```

17. Count set bits (1s) in the binary representation of a number.

```
def count_set_bits(n):
    count = 0
    while n:
        count += n & 1
        n >>= 1
    return count
```

18. Find the longest substring without repeating characters (sliding window).

```
def longest_unique_substr(s):
    seen = {}
    start = max_len = 0
    for i, ch in enumerate(s):
        if ch in seen and seen[ch] >= start:
            start = seen[ch] + 1
        seen[ch] = i
        max_len = max(max_len, i - start + 1)
    return max_len
```

19. Matrix transpose.

```
def transpose(matrix):
    return [list(row) for row in zip(*matrix)]
```

20. Check Armstrong number (e.g., $153 = 1^3 + 5^3 + 3^3$).

```
def is_armstrong(n):
    digits = str(n)
    power = len(digits)
    return n == sum(int(d) ** power for d in digits)
```

Language choice tip: Python reads fastest for Accenture-style array/string tasks and needs the least boilerplate under time pressure — use it unless your placement drive restricts language choice or you are far more fluent in Java/C#.

11.5 Coding Round Time Management

- Read the full problem and constraints before writing a single line.
- Hand-trace the given example against your understanding (Section 11.2 shows why this matters).
- Write the brute-force version first if you are stuck — partial credit exists for passing some test cases.
- Test edge cases mentally: empty array, single element, all same values, negative numbers, $i=0$.
- Do not over-engineer; Accenture rewards a correct, readable solution over a clever one.

12. Round 3 — SQL Coding Question

Q2 of the coding round gives you a schema (the deck example has 6 tables) and asks for one non-trivial query combining joins, filtering, aggregation and sorting.

12.1 Solved: the deck's "High-Value E-Commerce Customers" question

Problem: Retrieve customer_name, total number of completed orders, and total amount spent, for customers who spent more than \$1,000 in 2025 across all completed orders shipped to 'USA'. Sort by total amount spent, descending.

Schema: Customers(customer_id, customer_name, country_id) • Orders(order_id, customer_id, order_date, status_id) • Order_Items(item_id, order_id, product_id, quantity, unit_price) • Products(product_id, product_name, category_id) • Order_Status(status_id, status_name) • Countries(country_id, country_name)

Step-by-step construction

1. Start from Customers, join to Orders (customer_id), join to Order_Items (order_id), join to Order_Status (status_id) and Countries (country_id).
2. Filter: status_name = 'Completed', country_name = 'USA', YEAR(order_date) = 2025.
3. Compute per-customer: COUNT(DISTINCT order_id) for orders, SUM(quantity*unit_price) for amount.
4. Group by customer, filter the group with HAVING (aggregate can't go in WHERE), then order.

```
SELECT
  c.customer_name,
  COUNT(DISTINCT o.order_id)      AS total_completed_orders,
  SUM(oi.quantity * oi.unit_price) AS total_amount_spent
FROM Customers c
JOIN Orders o      ON c.customer_id = o.customer_id
JOIN Order_Items oi ON o.order_id   = oi.order_id
JOIN Order_Status os ON o.status_id = os.status_id
JOIN Countries ct  ON c.country_id = ct.country_id
WHERE os.status_name = 'Completed'
      AND ct.country_name = 'USA'
      AND YEAR(o.order_date) = 2025
GROUP BY c.customer_id, c.customer_name
HAVING SUM(oi.quantity * oi.unit_price) > 1000
ORDER BY total_amount_spent DESC;
```

Common mistakes on this exact style of question: (1) Filtering on total spend in WHERE instead of HAVING — it must be HAVING because it's an aggregate. (2) Using COUNT(o.order_id) instead of COUNT(DISTINCT o.order_id) — since Order_Items creates one row per item, a customer's order gets counted multiple times without DISTINCT. (3) Forgetting to GROUP BY both customer_id and customer_name (some engines require every non-aggregated SELECT column in GROUP BY). (4) Joining Products/category_id when the question doesn't need them — unnecessary joins that don't 1:1 match can silently duplicate rows.

12.2 More Practice: Multi-table Aggregation Questions

```
-- Q: Departments where average salary exceeds company average
SELECT d.dept_name, AVG(e.salary) avg_sal
FROM employees e JOIN departments d ON e.dept_id = d.dept_id
GROUP BY d.dept_name
HAVING AVG(e.salary) > (SELECT AVG(salary) FROM employees);

-- Q: Students who scored above average in every subject they took
SELECT s.student_name
FROM scores sc JOIN students s ON sc.student_id = s.student_id
GROUP BY s.student_id, s.student_name
HAVING MIN(sc.marks) > (SELECT AVG(marks) FROM scores);

-- Q: Products never ordered (anti-join)
SELECT p.product_name FROM products p
LEFT JOIN order_items oi ON p.product_id = oi.product_id
WHERE oi.product_id IS NULL;

-- Q: Top spender per country (window function)
SELECT country, customer_name, total_spent FROM (
  SELECT c.country, c.customer_name, SUM(o.amount) total_spent,
         RANK() OVER (PARTITION BY c.country ORDER BY SUM(o.amount) DESC) rnk
  FROM customers c JOIN orders o ON c.customer_id = o.customer_id
  GROUP BY c.country, c.customer_name) t
WHERE rnk = 1;
```

13. Round 3 — Web Development (HTML/CSS/JS)

Q3 usually gives you a starter HTML/JS snippet with a bug or missing feature and asks you to modify JavaScript to meet stated rules.

13.1 Solved: the deck's "Interactive Counter with Limit" question

Problem: Modify the given JavaScript so that: (1) Increment cannot exceed 10, (2) Decrement cannot go below 0, (3) a paragraph with id `status-msg` shows "Maximum limit reached!" at 10, "Minimum limit reached!" at 0, and is empty otherwise.

```
<!-- HTML -->
<p id="count-display">0</p>
<button id="increment">Increment</button>
<button id="decrement">Decrement</button>
<p id="status-msg"></p>

<script>
let count = 0;
const MIN = 0, MAX = 10;
const display = document.getElementById('count-display');
const status = document.getElementById('status-msg');

function updateUI() {
  display.textContent = count;
  if (count === MAX) {
    status.textContent = "Maximum limit reached!";
  } else if (count === MIN) {
    status.textContent = "Minimum limit reached!";
  } else {
    status.textContent = "";
  }
}

document.getElementById('increment').addEventListener('click', () => {
  if (count < MAX) count++;
  updateUI();
});

document.getElementById('decrement').addEventListener('click', () => {
  if (count > MIN) count--;
  updateUI();
});

updateUI();
</script>
```

Why this passes every test case: the boundary check (`if (count < MAX)`) happens *before* the increment, so count physically cannot exceed 10 or go below 0 — you don't need to clamp afterward. `updateUI()` is called once at the end so the display and message never get out of sync, and it's also called on load so the initial state (0) correctly shows "Minimum limit reached!".

13.2 HTML/CSS/JS Cheat Notes

Topic	Key facts
Semantic HTML	<header> <nav> <main> <article> <section> <footer> instead of generic <div>
Box model	content → padding → border → margin. <code>box-sizing: border-box</code> includes padding/border in width.
Flexbox	<code>display:flex</code> ; <code>justify-content</code> ; <code>align-items</code> ; <code>flex-direction</code> ; <code>gap</code>
Grid	<code>display:grid</code> ; <code>grid-template-columns</code> ; <code>grid-gap</code>
Selectors specificity	inline > ID > class/attribute > element
var vs let vs const	var: function-scoped, hoisted. let: block-scoped, reassignable. const: block-scoped, not reassignable (but object contents can mutate).
== vs ===	== does type coercion; === compares type and value, always prefer it
Arrow functions	No own <code>this</code> binding — inherits from enclosing scope
DOM selection	<code>getElementById</code> , <code>querySelector</code> , <code>querySelectorAll</code>
Event handling	<code>addEventListener('click', fn)</code> ; <code>event.preventDefault()</code> stops default action; <code>event.stopPropagation()</code> stops bubbling
Array methods	<code>map</code> (transform), <code>filter</code> (select), <code>reduce</code> (aggregate), <code>forEach</code> (iterate, no return), <code>find</code> , <code>some</code> , <code>every</code>
Promises / async	<code>.then().catch()</code> chains; <code>async/await</code> is syntactic sugar over Promises
JSON	<code>JSON.stringify(obj) → string</code> ; <code>JSON.parse(str) → object</code>
Hoisting	var and function declarations are hoisted to the top of scope; let/const are hoisted but stay in the "temporal dead zone"
Closures	An inner function retains access to its outer function's variables even after the outer function returns

13.3 Practice Task: Live Character Counter

Task: A textarea and a counter showing "X / 100 characters" that turns red when the limit is exceeded and disables the submit button.

```

<textarea id="msg" maxLength="150"></textarea>
<p id="counter">0 / 100 characters</p>
<button id="submit">Submit</button>

<script>
const msg = document.getElementById('msg');
const counter = document.getElementById('counter');
const submitBtn = document.getElementById('submit');
const LIMIT = 100;

msg.addEventListener('input', () => {
  const len = msg.value.length;
  counter.textContent = `${len} / ${LIMIT} characters`;
  const overLimit = len > LIMIT;
  counter.style.color = overLimit ? 'red' : 'black';
  submitBtn.disabled = overLimit;
});
</script>

```

13.4 Practice Task: Toggle a To-Do Item as Complete

```

<ul id="list">
  <li>Buy milk</li>
  <li>Finish assignment</li>
</ul>

<style>
.done { text-decoration: line-through; color: gray; }
</style>

<script>
document.getElementById('list').addEventListener('click', (e) => {
  if (e.target.tagName === 'LI') {
    e.target.classList.toggle('done');
  }
});
</script>

```

Event delegation trick: attaching one listener to the parent `<ul>` instead of each `<li>` means new items added later still work automatically — a pattern Accenture web questions frequently reward.

14. Communication Assessment

Runs as its own stage (separate from the written test), and on many drives it comes **first**. It is AI-graded (Accenture's own tool, not Versant/Pearson, though the format is comparable). ~**20-63 questions depending on the drive, 20-30 minutes**. No human evaluator is present, so pronunciation clarity, fluency and correct listening matter more than raw vocabulary.

14.1 Section-wise Breakdown

Section	Format	What is scored
Read Aloud	Text appears on screen, you read it aloud	Pronunciation, pace, fluency, stress
Repeat Sentence	You hear a sentence once, then repeat it exactly	Listening accuracy, memory, pronunciation
Reading Comprehension	Passage + MCQs	Comprehension, vocabulary
Sentence Completion / Grammar	Fill in the correct word/tense	Grammar accuracy
Story/Passage Retelling	Hear or read a short story, retell in your own words	Fluency, structure, vocabulary range
Open-ended Spoken Response	Answer a prompt in 30-45 seconds	Spontaneous fluency, coherence, confidence

14.2 How the AI Grader Actually Scores You

- **Fluency:** minimal pauses, filler words ("umm", "like") reduce score — speak in complete thought-chunks.
- **Pronunciation:** clear consonants at word endings matter more than "accent". Don't imitate a foreign accent — speak your natural English clearly and a little slower than casual conversation.
- **Pace:** too fast blurs words together; too slow reads as hesitant. Aim for calm, newsreader pace.
- **Content relevance** (for open-ended items): answer the actual prompt, do not go off-topic even if fluent.
- **Timing:** the mic starts recording immediately after the prompt — do not wait in silence; start within 1-2 seconds.

14.3 Preparation Drills

1. **Record yourself daily** reading a news paragraph aloud, then play it back. You will hear your own filler words and mumbled endings.
2. **Shadow native audio** (BBC/TED clips): pause every sentence and repeat it exactly, matching rhythm.
3. **Practice the 45-second answer structure:** 1 sentence stating your view → 2-3 sentences of reasoning/example → 1 sentence closing. This structure alone prevents rambling and running out of things to say.
4. **Test environment:** quiet room, decent headset mic, stable internet, sit upright (posture affects breath support and clarity).
5. **Never go silent.** If you don't know a word, paraphrase — a fluent wrong word beats a long silent pause.

14.4 Sample Open-Ended Prompts to Practice

- "Describe a time you solved a difficult problem." (Answer using: Situation → Action → Result, in under 45 seconds.)
- "Do you prefer working independently or in a team? Why?"
- "Describe your hometown/college in a few sentences."
- "What is one skill you want to develop in the next year?"

Trap: Do not memorise a scripted answer word-for-word — the AI can detect unnaturally uniform pacing/intonation typical of recitation, and memorised answers fall apart the moment the prompt is slightly different from what you rehearsed. Practice the *structure*, not a script.

15. Technical Interview — Question & Answer Bank

Format: 20–30 minutes, one or two interviewers, mix of resume-based, fundamentals, and a light live-coding or whiteboard question.

15.1 "Tell me about yourself" — the template

30–45 seconds. Structure: **(1) Who you are academically** — degree, college, key strength → **(2) What you've built/done** — 1–2 projects or experiences most relevant to the role → **(3) Why this role/Accenture** — connect your interest to what they do. Do not repeat your entire resume line by line.

15.2 General & CS Fundamentals (Q&A)

Question	Model answer approach
Explain OOP with a real-world example.	Use a Car/Vehicle hierarchy: encapsulation = private fuel level with getFuel(); inheritance = ElectricCar extends Vehicle; polymorphism = start() behaves differently per vehicle type; abstraction = the driver doesn't need engine internals.
What is the difference between process and thread?	Process = independent execution unit with its own memory; thread = lightweight unit within a process sharing memory. Give an example: a browser is a process, each tab/download can be a thread.
Explain normalization and why it matters.	Reduces redundancy and update anomalies by organizing data into related tables via 1NF/2NF/3NF; trade-off is more joins at query time.
What happens when you type a URL and press Enter?	DNS resolution → TCP handshake → TLS handshake (if HTTPS) → HTTP request → server processes → response → browser renders HTML/CSS/JS.
What is an API? Have you used one?	A contract that lets two systems communicate. Mention REST, HTTP methods (GET/POST/PUT/DELETE), status codes, and any project where you consumed or built one.
Explain the SDLC.	Requirement gathering → Design → Implementation → Testing → Deployment → Maintenance. Mention Agile/Scrum if relevant to your project experience.
Difference between Agile and Waterfall?	Waterfall: sequential, rigid, plan upfront. Agile: iterative sprints, adapts to change, continuous feedback.
What is cloud computing, in your own words?	On-demand delivery of computing resources over the internet, paid per use, without owning physical hardware. Mention IaaS/PaaS/SaaS if asked to go deeper.
What is version control and why use Git?	Tracks changes over time, enables collaboration, branching for parallel work, and rollback if something breaks.
Explain a project from your resume, end to end.	Problem → your specific role → tech stack → one challenge you solved → the outcome/impact. Be ready for a follow-up "why did you choose X over Y".
What is the difference between SQL and NoSQL?	SQL: structured schema, relational, ACID (MySQL, PostgreSQL). NoSQL: flexible schema, horizontal scaling, eventual consistency (MongoDB, Cassandra). Choose based on data structure and scale needs.
What design pattern have you used or know about?	Singleton (one instance, e.g. a config/logger), Factory (creates objects without exposing instantiation logic), Observer (pub-sub, e.g. event listeners).
How do you handle exceptions in your preferred language?	try-catch-finally; catch specific exceptions before generic ones; never swallow exceptions silently; log meaningfully.
What is the time complexity of binary search, and why?	$O(\log n)$ — each comparison halves the search space; requires a sorted array.
How would you find duplicate elements in an array efficiently?	HashSet/HashMap for $O(n)$ time $O(n)$ space, versus nested loop $O(n^2)$ with no extra space — state the trade-off.

15.3 Live-Coding / Whiteboard Readiness

- Expect one of your Chapter 11 practice problems, sometimes slightly modified.
- **Think aloud** — interviewers evaluate your reasoning process, not just the final answer.
- State the brute-force approach first, then optimize if time allows; interviewers like seeing you improve incrementally.
- Always state time and space complexity of your final solution.
- Ask a clarifying question before coding if the problem has any ambiguity (e.g., "can the array have negative numbers?").

16. HR Interview — Question & Answer Bank

Question	Approach / model answer points
Why do you want to join Accenture?	Mention its scale in consulting + technology + global delivery, exposure to diverse client domains, and learning platforms (e.g. LearnVantage) — tie it to your own growth goals, not generic praise.
Where do you see yourself in 5 years?	A believable growth path: deepening technical expertise, taking on more ownership/leadership, staying within the tech/consulting space — avoid naming a specific senior title.
What are your strengths?	Pick 2, each backed by a specific one-line example (not just the word).
What are your weaknesses?	A real, minor weakness + the concrete action you're taking to improve it. Avoid "I'm a perfectionist" clichés.
Tell me about a time you failed.	STAR method: Situation, Task, Action, Result — end with what you learned and how you apply it now.
Tell me about a conflict with a teammate and how you resolved it.	Focus on communication and compromise, not blame. End with a positive outcome.
Are you comfortable relocating / working in shifts?	Answer honestly but align with the role's requirement — Accenture fresher roles usually require flexibility on location and shift.
Why should we hire you?	Connect 2–3 of your genuine strengths directly to what the role needs — be specific, not generic ("I'm hardworking").
Do you have any questions for us?	Always have 1–2 ready: about the team you'd join, learning opportunities, or what a typical first project looks like. Never say "no".
How do you handle pressure/deadlines?	Give a specific method (prioritization, breaking into subtasks, communicating early if at risk) with a brief real example.
What do you know about Accenture?	Global professional services company spanning strategy, consulting, technology and operations; serves clients across industries; strong focus on digital, cloud and AI transformation currently.
Describe yourself in three words.	Pick words you can each justify with one sentence if asked "why".
What motivates you?	Be specific and genuine — learning new things, solving real problems, seeing impact of your work, etc.
Any backlogs / gap in education? Explain.	Be honest, brief, and focus on the resolution/lesson, not excuses.
Salary expectations?	For freshers, defer to the company's standard package for the role/track; you can say you're open and focused on the learning opportunity.

STAR method (use for every behavioural HR question): **S**ituation (brief context) → **T**ask (what needed doing) → **A**ction (what *you* specifically did) → **R**esult (outcome, ideally with a number or a lesson learned). Keep the whole answer to 60–90 seconds.

17. Master Cheat Sheet (One-Page Revision)

Pseudocode

- $\wedge$ = XOR (not power!)
- `do-while` runs ≥ 1 time
- Integer division truncates: $7/2=3$
- $a^a=0$, $a^0=a$, $a^b^a=b$
- $n \ll k = n \times 2^k$; $n \gg k = n/2^k$ (floor)

OOPs

- 4 pillars: Encapsulation, Abstraction, Inheritance, Polymorphism
- Overloading = compile-time, same class
- Overriding = runtime, parent-child, same signature
- static/final/private methods cannot be overridden
- Java: no multiple class inheritance (Diamond Problem)

SQL / DBMS

- Order: FROM $\rightarrow$ JOIN $\rightarrow$ WHERE $\rightarrow$ GROUP BY $\rightarrow$ HAVING $\rightarrow$ SELECT $\rightarrow$ ORDER BY
- WHERE filters rows, HAVING filters groups
- DELETE=DML/rollback-able; TRUNCATE=DDL/fast; DROP=removes table
- Primary key: unique+not null; Foreign key: references another table
- 1NF atomic, 2NF no partial dep, 3NF no transitive dep

OS

- Deadlock needs all 4: Mutual exclusion, Hold&wait, No preemption, Circular wait
- SJF = min avg wait; RR = best for time-sharing
- Paging $\rightarrow$ internal fragmentation; Segmentation $\rightarrow$ external
- Thrashing = too much paging, too little real work
- Semaphore = counter/signal; Mutex = lock/ownership

Networking

- OSI 7 layers:
Physical, Data Link, Network, Transport, Session, Presentation, Application
- TCP = reliable, connection-oriented; UDP = fast, connectionless
- Ports: 80 HTTP, 443 HTTPS, 22 SSH, 53 DNS, 25 SMTP
- DHCP assigns IP; DNS resolves names; NAT maps private $\leftrightarrow$ public

Security

- CIA Triad: Confidentiality, Integrity, Availability
- Symmetric: AES/DES (1 key). Asymmetric: RSA (2 keys)
- Digital signature: sign with private key, verify with public key
- SQLi $\rightarrow$ use prepared statements; XSS $\rightarrow$ sanitize output

Wi-Fi & Cloud

- WEP broken, WPA weak, WPA2 good, **WPA3 best (SAE)**
- IaaS (infra) < PaaS (platform) < SaaS (software) — increasing abstraction
- Shared Responsibility: provider secures cloud, customer secures data/config

DevOps

- CI = integrate+test often; CD = deploy often
- Docker: image $\rightarrow$ container. Kubernetes: pod $\rightarrow$ deployment $\rightarrow$ service
- Terraform = provisioning (IaC); Ansible = configuration
- Blue-Green = 2 environments switch; Canary = gradual rollout

Cognitive Games

- Maze: map walls, don't just wander — fewest attempts wins
- Bubble math: read the sort direction first, estimate don't compute exactly
- Path Finder: fix exit tile first, work backward

Exam Numbers to Remember

- Written test: 125 Q / 175 min, 4 sections
- Behavioural: 54 Q / 20 min
- Cognitive games: 3–4 games / 20 min
- Technical MCQ: 45 Q / 45 min
- Coding: 3 Q (DSA, SQL, Web) / 60 min

18. Exam-Day Playbook & Common Mistakes

18.1 The Night Before

- Test your webcam, mic, and internet connection with the exact browser you'll use (Chrome recommended unless told otherwise).
- Close all other tabs/apps; disable notifications; keep your ID and any required documents ready.
- Sleep on time — the communication test especially punishes tired, slurred speech.
- Re-skim this guide's Chapter 17 cheat sheet, not the full guide.

18.2 During the Test

- **Answer everything.** Sections are usually sectional and non-reversible — there is no "come back later" across sections, and often not within a section either.
- **Behavioural round:** keep a steady pace, ~20 seconds/question, don't overthink.
- **Cognitive games:** prioritize learning from failed attempts over speed; the process is scored.
- **Technical MCQ:** do the topics you're strongest in first to bank sectional-cutoff marks, then attempt the rest.
- **Coding:** read constraints fully, hand-trace the sample before coding, submit something for every question — partial credit exists.
- **Communication test:** speak clearly and immediately after each prompt; don't restart mid-answer even if you stumble — recover and continue.

18.3 Top 10 Mistakes That Cost Marks

1. Treating $\wedge$ as exponentiation instead of XOR in pseudocode.
2. Filtering an aggregate in WHERE instead of HAVING in SQL.
3. Confusing DELETE, TRUNCATE, and DROP.
4. Not checking the sort direction ("lowest to highest" vs reverse) in bubble-math games.
5. Answering "Highly Agree" to almost every behavioural statement (looks fake).
6. Leaving coding questions blank instead of submitting a partial/brute-force solution.
7. Not hand-tracing the sample input/output before coding — leads to logic that "looks right" but fails hidden test cases.
8. Going silent for several seconds in the communication test instead of recovering mid-sentence.
9. Memorizing a scripted HR/communication answer word-for-word instead of practicing structure.
10. Spending too long perfecting one MCQ instead of banking easy marks across the section first.

18.4 Final Words

Accenture's fresher hiring is designed to be passable by a well-prepared, average student — it does not require competitive-programming-level skill. The candidates who clear it are the ones who treat **pseudocode tracing, SQL joins, and basic coding patterns as drilled habits**, not one-time reading. Revise Chapter 17 the morning of your test, trust your preparation, and manage the clock. All the best.

This guide is compiled from a faculty-prepared SRMIST slide deck plus publicly available 2026 placement-preparation sources (PrepInsta, FaceGP/FACE Prep, GUVI, Jobaaj Learnings, and others). Exact patterns, question counts and cut-offs vary by drive, region, and year — always cross-check with your own invitation email and placement cell.